

ITM Skills University – B.Tech

DBMS Assignment 1 – Core SQL DDL & DML

College Database

Student Name	Sasanka Sekhar Kundu
Subject	Database Management System
Database	PostgreSQL – college_assignment_1

1. Assignment Objective

The objective of this assignment is to practice the core SQL commands used in Data Definition Language (DDL) and Data Manipulation Language (DML). A CollegeStudents table is created and then populated, queried, updated and modified using CREATE, INSERT, WHERE, UPDATE, DELETE and the MAX() aggregate function.

2. Table Structure

Column	Data Type	Description
StudentID	INT	Unique identification number for each student
StudentName	VARCHAR(100)	Name of the student
Department	VARCHAR(100)	Department in which the student is enrolled
Age	INT	Age of the student
Marks	INT	Marks obtained by the student

3. Question 1 – CREATE TABLE

Create a table named CollegeStudents with the required columns.

```
CREATE TABLE CollegeStudents (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(100) NOT NULL,  
    Department VARCHAR(100) NOT NULL,  
    Age INT NOT NULL,  
    Marks INT NOT NULL  
);
```

```

'demo_assignment_1=# \c postgres
You are now connected to database "postgres" as user "zoro".
postgres=# CREATE DATABASE college_assignment_1;
CREATE DATABASE
postgres=# \c college_assignment_1
You are now connected to database "college_assignment_1" as user "zoro".
college_assignment_1=# CREATE TABLE CollegeStudents (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    Department VARCHAR(100) NOT NULL,
    Age INT NOT NULL,
    Marks INT NOT NULL
);
CREATE TABLE
college_assignment_1=# \dt
      List of tables
Schema | Name      | Type  | Owner
-----|-----|-----|-----
public | collegestudents | table | zoro
(1 row)

college_assignment_1=# INSERT INTO CollegeStudents (
    StudentID,
    StudentName,
    Department,
    Age,
    Marks
)
VALUES (
    101,
    'Rahul',
    'Computer Science',
    19,
    85
);
INSERT 0 1
college_assignment_1=# INSERT INTO CollegeStudents (
    StudentID,
    StudentName,
    Department,
    Age,
    Marks
)
VALUES (
    (102, 'Priya', 'Electronics', 20, 78),
    (103, 'Aman', 'Mechanical', 21, 35),
    (104, 'Sneha', 'Information Technology', 19, 92);
INSERT 0 3
college_assignment_1=# SELECT * FROM CollegeStudents;
 studentid | studentname | department      | age | marks
-----|-----|-----|-----|-----
      101 | Rahul       | Computer Science |  19 |    85
      102 | Priya       | Electronics      |  20 |    78
      103 | Aman        | Mechanical       |  21 |    35
      104 | Sneha       | Information Technology |  19 |    92
(4 rows)

college_assignment_1=# SELECT *
FROM CollegeStudents
WHERE Department = 'Computer Science';
 studentid | studentname | department      | age | marks
-----|-----|-----|-----|-----
      101 | Rahul       | Computer Science |  19 |    85
(1 row)

college_assignment_1=# UPDATE CollegeStudents
SET Marks = 90
WHERE StudentID = 101;
UPDATE 1

```

SCREENSHOT 1 – CREATE TABLE SUCCESS

4. Question 2 – INSERT Rahul

Insert the first student record.

```

INSERT INTO CollegeStudents (
    StudentID,
    StudentName,
    Department,
    Age,
    Marks
)
VALUES (
    101,
    'Rahul',
    'Computer Science',
    19,
    85
);

```

5. Question 3 – INSERT Three More Students

Insert the remaining three student records.

```

INSERT INTO CollegeStudents (
    StudentID,
    StudentName,
    Department,

```

```

Age,
Marks
)
VALUES
(102, 'Priya', 'Electronics', 20, 78),
(103, 'Aman', 'Mechanical', 21, 35),
(104, 'Sneha', 'Information Technology', 19, 92);

```

```

postgres=# \c college_assignment_1
You are now connected to database "college_assignment_1" as user "zero".
college_assignment_1=# CREATE TABLE CollegeStudents (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    Department VARCHAR(100) NOT NULL,
    Age INT NOT NULL,
    Marks INT NOT NULL
);
college_assignment_1=# \dt
List of tables
Schema | Name | Type | Owner
public | collegestudents | table | zero
(1 row)

college_assignment_1=# INSERT INTO CollegeStudents (
    StudentID,
    StudentName,
    Department,
    Age,
    Marks
)
VALUES (
    101,
    'Rahul',
    'Computer Science',
    19,
    85
);
INSERT 0 1
college_assignment_1=# INSERT INTO CollegeStudents (
    StudentID,
    StudentName,
    Department,
    Age,
    Marks
)
VALUES
(102, 'Priya', 'Electronics', 20, 78),
(103, 'Aman', 'Mechanical', 21, 35),
(104, 'Sneha', 'Information Technology', 19, 92);
INSERT 0 3
college_assignment_1=# SELECT * FROM CollegeStudents;
 studentid | studentname | department | age | marks
-----
101 | Rahul | Computer Science | 19 | 85
102 | Priya | Electronics | 20 | 78
103 | Aman | Mechanical | 21 | 35
104 | Sneha | Information Technology | 19 | 92
(4 rows)

college_assignment_1=# SELECT *
FROM CollegeStudents
WHERE Department = 'Computer Science';
 studentid | studentname | department | age | marks
-----
101 | Rahul | Computer Science | 19 | 85
(1 row)

college_assignment_1=# UPDATE CollegeStudents
SET Marks = 90
WHERE StudentID = 101;
UPDATE 1
college_assignment_1=# SELECT * FROM CollegeStudents
WHERE StudentID = 101;
 studentid | studentname | department | age | marks
-----
101 | Rahul | Computer Science | 19 | 90

```

SCREENSHOT 2 – INSERT SUCCESS.

6. Question 4 – WHERE: Computer Science Students

```

SELECT *
FROM CollegeStudents
WHERE Department = 'Computer Science';

```

Expected result: Rahul is displayed because his initial department is Computer Science.

SCREENSHOT 3 – WHERE QUERY RESULT

7. Question 5 – UPDATE Rahul's Marks

```

UPDATE CollegeStudents
SET Marks = 90
WHERE StudentID = 101;

SELECT *
FROM CollegeStudents
WHERE StudentID = 101;

```

The marks of Rahul are updated from 85 to 90.

8. Question 6 – UPDATE Rahul's Department

```
UPDATE CollegeStudents
SET Department = 'Information Technology'
WHERE StudentName = 'Rahul';

SELECT *
FROM CollegeStudents
WHERE StudentName = 'Rahul';
```

```
college_assignment_1=# UPDATE CollegeStudents
SET Marks = 90
[WHERE StudentID = 101;
UPDATE 1
college_assignment_1=# SELECT * FROM CollegeStudents
[WHERE StudentID = 101;
 studentid | studentname | department | age | marks
-----+-----+-----+-----+-----
      101 | Rahul      | Computer Science | 19 | 90
(1 row)

college_assignment_1=# UPDATE CollegeStudents
SET Department = 'Information Technology'
[WHERE StudentName = 'Rahul';
UPDATE 1
college_assignment_1=# SELECT *
FROM CollegeStudents
[WHERE StudentName = 'Rahul';
 studentid | studentname | department | age | marks
-----+-----+-----+-----+-----
      101 | Rahul      | Information Technology | 19 | 90
(1 row)
```

SCREENSHOT 4 – UPDATE SUCCESS

9. Question 7 – DELETE StudentID 103

```
DELETE FROM CollegeStudents
WHERE StudentID = 103;
```

This deletes Aman from the table.

10. Question 8 – DELETE Students with Marks Less Than 40

```
DELETE FROM CollegeStudents
WHERE Marks < 40;
```

Because Aman, who had 35 marks, was already deleted in Question 7, this query may return DELETE 0. This is correct for the required execution order.

```
college_assignment_1=# DELETE FROM CollegeStudents
[WHERE StudentID = 103;
DELETE 1
college_assignment_1=# DELETE FROM CollegeStudents
[WHERE Marks < 40;
DELETE 0
college_assignment_1=# SELECT StudentName
FROM CollegeStudents
[WHERE Marks > 75;
studentname
-----
Priya
Sneha
Rahul
(3 rows)
```

SCREENSHOT 5 – DELETE OPERATIONS

11. Question 9 – Students with Marks Greater Than 75

```
SELECT StudentName
FROM CollegeStudents
WHERE Marks > 75;
```

Expected final names: Rahul, Priya and Sneha.

12. Question 10 – Aggregate Function: Highest Marks

```
SELECT MAX(Marks) AS Highest_Marks
FROM CollegeStudents;
```

Expected result: 92.

```

(4 rows)

college_assignment_1=# SELECT *
FROM CollegeStudents
WHERE Department = 'Computer Science';
 studentid | studentname | department | age | marks 
-----
 101 | Rahul | Computer Science | 19 | 85
(1 row)

college_assignment_1=# UPDATE CollegeStudents
SET Marks = 90
WHERE StudentID = 101;
UPDATE 1
college_assignment_1=# SELECT * FROM CollegeStudents
WHERE StudentID = 101;
 studentid | studentname | department | age | marks 
-----
 101 | Rahul | Computer Science | 19 | 90
(1 row)

college_assignment_1=# UPDATE CollegeStudents
SET Department = 'Information Technology'
WHERE StudentName = 'Rahul';
UPDATE 1
college_assignment_1=# SELECT *
FROM CollegeStudents
WHERE StudentName = 'Rahul';
 studentid | studentname | department | age | marks 
-----
 101 | Rahul | Information Technology | 19 | 90
(1 row)

college_assignment_1=# DELETE FROM CollegeStudents
WHERE StudentID = 103;
DELETE 1
college_assignment_1=# DELETE FROM CollegeStudents
WHERE Marks < 40;
DELETE 0
college_assignment_1=# SELECT StudentName
FROM CollegeStudents
WHERE Marks > 75;
 studentname 
-----
 Priya
 Sneha
 Rahul
(3 rows)

college_assignment_1=# SELECT MAX(Marks) AS Highest_Marks
FROM CollegeStudents;
 highest_marks 
-----
 92
(1 row)

college_assignment_1=# \dt
      List of tables
Schema | Name          | Type  | Owner
-----
 public | collegestudents | table | zoro
(1 row)

college_assignment_1=# █

```

SCREENSHOT 6 – FINAL SELECT AND MAX() RESULT

13. Final Table State

StudentID	StudentName	Department	Age	Marks
101	Rahul	Information Technology	19	90
102	Priya	Electronics	20	78
104	Sneha	Information Technology	19	92

14. Complete SQL Code

```

-- Question 1
CREATE TABLE CollegeStudents (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(100) NOT NULL,
    Department VARCHAR(100) NOT NULL,
    Age INT NOT NULL,
    Marks INT NOT NULL
);

-- Question 2
INSERT INTO CollegeStudents
VALUES (101, 'Rahul', 'Computer Science', 19, 85);

-- Question 3
INSERT INTO CollegeStudents
VALUES

```

```

    (102, 'Priya', 'Electronics', 20, 78),
    (103, 'Aman', 'Mechanical', 21, 35),
    (104, 'Sneha', 'Information Technology', 19, 92);

-- Question 4
SELECT * FROM CollegeStudents
WHERE Department = 'Computer Science';

-- Question 5
UPDATE CollegeStudents
SET Marks = 90
WHERE StudentID = 101;

-- Question 6
UPDATE CollegeStudents
SET Department = 'Information Technology'
WHERE StudentName = 'Rahul';

-- Question 7
DELETE FROM CollegeStudents
WHERE StudentID = 103;

-- Question 8
DELETE FROM CollegeStudents
WHERE Marks < 40;

-- Question 9
SELECT StudentName
FROM CollegeStudents
WHERE Marks > 75;

-- Question 10
SELECT MAX(Marks) AS Highest_Marks
FROM CollegeStudents;

```

15. Conclusion

This assignment demonstrates the complete basic SQL workflow. The CollegeStudents table was created using CREATE TABLE, records were added using INSERT, specific records were displayed using WHERE, existing values were modified using UPDATE, unwanted records were removed using DELETE, and the highest marks were identified using the MAX() aggregate function. The final table contains Rahul, Priya and Sneha, while Aman was removed according to the assignment requirements.